

Preference-Transition Multi-LoRA Mamba System for Sequential Recommendation

Zhi-Quan Feng

Abstract

Sequential recommendation must represent both stable interests and abrupt changes in user intent. This report presents the independent ver4 Preference-Transition Multi-LoRA Mamba system implemented in `MediaTek_2026_project`. It fuses cached representations from a frozen Mamba-2.8B text encoder with trainable two-layer LightGCN item embeddings constructed exclusively from training interactions. Four functional agents—long-horizon, short-horizon, preference-transition, and coordinator—share this item space but use disjoint, automatically routed LoRA banks. The temporal agents model complementary time scales; the preference agent discovers latent prototypes, infers the current soft preference distribution, predicts the next distribution, and estimates preference-shift probability; and the coordinator combines all candidate scores with learned temporal weights and a context-dependent preference gate. The Mamba-2.8B backbone is never fine-tuned and is not duplicated per agent.

Training uses specialist warm-up, coordinator warm-up, and joint multi-step listwise ranking. Each epoch resamples bounded training prefixes and mines hard negatives from a larger model-scored random pool. Preference learning adds next-distribution KL divergence, soft transition detection, prototype balance, prototype separation, and joint-stage contrastive alignment. Validation and test ranking always cover the complete catalog.

1 Introduction

Recommender systems have traditionally been formulated as rating prediction, click prediction, or next-item ranking problems. These formulations are effective when the objective is immediate relevance, but they do not fully represent the interactive nature of recommendation. A recommendation changes what the user sees, the resulting response changes the observable user state, and that new state influences subsequent recommendations. Reinforcement learning (RL) offers a natural language for this process by representing the recommender as a policy, the user as an environment, and engagement or satisfaction as a reward [?].

The promise of RL does not make its application straightforward. Industrial recommenders operate over thousands or millions of discrete actions, whereas most deep RL algorithms were developed for much smaller action spaces. Real user exploration is costly, so policies are usually trained from logged data generated by an earlier recommender. Feedback is incomplete: a missing click does not prove that an unseen item was irrelevant. A reward such as click-through rate may also favor immediate attraction even when the intended objective is long-term satisfaction, retention, or diversity. These issues make policy design, reward design, and offline evaluation inseparable parts of an RL recommender.

Sequential user representation creates another challenge. A user can exhibit a stable preference over months while simultaneously pursuing a short-lived intent within the current session. A single recurrent state must decide how quickly to forget old evidence, yet one global time scale is unlikely to fit both behaviors. Transformer recommenders can model long dependencies, but their self-attention cost grows quadratically with sequence length. Selective state-space models such as Mamba instead update a recurrent state in linear time and make the update input-dependent [?]. This mechanism is attractive for recommendation because an interaction can be selectively retained or forgotten according to its content.

A direct multi-agent adaptation of a large Mamba language model would be computationally inefficient. If every agent executed an independent 2.8-billion-parameter model for each candidate-ranking update, training would be dominated by repeated text encoding. Duplicating the backbone would also undermine the parameter-efficiency motivation. The central systems question is therefore not simply whether Mamba can replace a Transformer, but how its semantic and selective-state capabilities can be used in a shared multi-LoRA system without placing the foundation model inside the high-frequency optimization loop.

This project studies a fundamental question: *Can a shared Mamba semantic space, time-specialized rankers, and explicit preference-transition learning provide an efficient and interpretable foundation for sequential recommendation?*

The proposed answer is a four-agent multi-LoRA architecture. A frozen Mamba model first converts item metadata into a cached semantic matrix. Long- and short-term agents process the sequence with different selective dynamics. The preference agent converts the same history into a trajectory over latent prototypes and predicts its next state. A coordinator observes both temporal specialists and adds gated preference evidence to the final catalog ranking. Each agent follows its own LoRA route while sharing the semantic and collaborative item representation. The implementation is isolated in `ver4` and does not import code or artifacts from `ver1`–`ver3`.

2 Related Works and Innovations

2.1 Reinforcement Learning for Recommendation

Early deep RL recommenders commonly formulated recommendation as a Markov decision process and learned action values or policies from user feedback. DRN used deep Q-learning for news recommendation and explicitly modeled future reward and user return behavior [?]. List-wise and page-wise methods expanded the action from a single item to a slate, but this introduced a combinatorial action space. SlateQ addressed this problem by decomposing slate value under assumptions about user choice [?]. Policy-gradient methods offered another route: the Top- K off-policy REINFORCE recommender scaled policy learning to a production catalog and corrected bias from logged behavior policies [?].

These studies establish the reasons to use RL—exploration, delayed effects, and long-term value—but they also expose a gap between research formulations and common offline experiments. Many sequential recommenders use a sampled next-item RL objective even though the evaluation contains no environment transition caused by the recommended action. Recent work showed that part of the apparent benefit can arise because the RL term acts as an auxiliary representation-learning objective rather than because it learns a genuinely long-horizon policy [?]. A credible implementation must therefore distinguish its current offline objective from claims about online long-term satisfaction.

The present pipeline adopts a conservative interpretation. Static MovieLens, Amazon, and Yelp logs provide no action-dependent environment response, so `ver4` is trained as an offline sequential ranker with multi-step listwise losses, not as an online reinforcement-learning policy. A genuine RL extension would require logged propensities, a simulator, or online interaction and is outside the current claim.

2.2 Mamba and Sequential Recommendation

Transformer-based sequential recommenders capture rich dependencies but require quadratic attention computation. Mamba introduces input-dependent state-space parameters and linear sequence processing [?]. Mamba4Rec applied selective state-space modeling to sequential recommendation [?], while SIGMA introduced bidirectional and gated mechanisms to improve contextual and short-term modeling [?]. These results show that Mamba is useful for sequential recommendation, but replacing a Transformer with a standard Mamba block does not solve RL credit assignment, preference time-scale separation, or foundation-model cost.

This project uses Mamba in two distinct roles. The frozen Mamba-2.8B language model provides item semantics from natural-language metadata. Lightweight Mamba-inspired diagonal selective recurrences then provide trainable sequence states. This separation preserves linear state updates without executing the language model at every transition.

The agents are not copies of Mamba-2.8B, and their LoRA modules are not attached to the frozen language-model backbone. They share one cached item-semantic space and apply separate low-rank residual updates inside compact recommendation modules. Independent adaptation is therefore retained while online memory and computation remain proportional to the 128-dimensional recommendation space.

2.3 Multi-Agent and Multi-Time-Scale Recommendation

A multi-agent recommender should contain more than several identically used neural modules. In the proposed architecture, the long-term agent observes up to 100 interactions, the short-term agent observes the latest 10, the preference agent models a distribution over 64 latent prototypes, and the coordinator produces the deployed score distribution. These agents have distinct functional roles and disjoint LoRA routes even though they share item representations.

This division reflects the fact that stable preferences and transient intents evolve at different rates. A user may consistently prefer one genre while temporarily searching within another category. A short-term specialist can react quickly, whereas a slowly changing specialist can resist noisy or accidental interactions. The coordinator performs context-dependent arbitration instead of using a fixed average.

LoRA represents a residual update with two low-rank matrices [?]. Giving each agent its own LoRA parameters creates separate adaptation subspaces at a small cost. The ver4 Amazon configuration uses rank $r = 16$, scale parameter $\alpha = 32$, and dropout 0.05. Zero-initializing the second matrix makes every adapter begin as a no-op and stabilizes staged training.

2.4 Why This Topic Must Be Investigated

Scientifically, the topic tests whether multi-agent specialization is more than extra parameter count. If long- and short-term agents receive identical inputs and objectives, they may collapse to nearly identical scoring functions. A useful architecture must create structural differences through observation horizon, state time scale, and separate listwise losses, then verify these choices through ablation.

The topic also clarifies what Mamba contributes. A pretrained Mamba can supply textual knowledge, but repeatedly invoking it inside the training loop may erase any throughput advantage. A compact recurrent model is efficient but may fail to exploit item metadata. Separating semantic precomputation from selective recommender-state updates provides a testable middle ground.

Catalog ranking must also remain practical. Training over the full catalog at every step is expensive, while random sampled negatives are often trivial. This pipeline mines hard negatives from a larger model-scored pool during training and always evaluates against the complete catalog.

Finally, explanation generation should not dominate ranking latency. The pipeline performs retrieval first and loads the generator only after evaluation. This creates a clear boundary between the ranker and its post-hoc verbalization.

2.5 Innovations and Contributions

The implementation makes eight contributions:

1. A cooperative four-agent recommender with long- and short-term selective-state specialists, a latent preference-transition agent, and a learned coordinator.

2. A multi-LoRA bank with 13 disjoint residual adapters—four for each temporal specialist, two for the preference agent, and three for the coordinator—automatically selected by the corresponding forward path.
3. Shared cached Mamba semantics that avoid repeated forward passes of the frozen 2.8B backbone during training and ranking.
4. Explicit identification and prediction of preference transitions using soft prototypes, a recurrent transition encoder, and Jensen–Shannon-derived transition targets.
5. Three-stage optimization separating specialist learning, coordinator calibration, and joint listwise adaptation, with per-epoch prefix resampling, future soft labels, hard negatives, and preference contrastive alignment.
6. Leakage-controlled catalog calibration combining neural scores with training-only popularity and first-order transition statistics.
7. A unified MovieLens, Amazon Reviews 2023, and Yelp 2019/2023 evaluation interface with chronological splitting, periodic validation, best-state restoration, and full-catalog ranking.
8. Preference analysis isolated into a machine-readable report containing current/next latent preferences, entropy, transition probability, and ranking weight.

3 Baselines

This section consolidates published MovieLens, Yelp, and Amazon results that are useful for positioning the proposed pipeline. The collection is protocol-aware. A dataset name is not a complete specification: papers apply different rating thresholds, k -core filters, sessionization rules, and chronological or random splits. Candidate construction is equally important. A score obtained by ranking one positive against sampled negatives is generally much higher than a score obtained by ranking the complete catalog. Consequently, the tables below are literature references, not a leaderboard across table boundaries.

Table 1: Organization of the external evidence. An exact match requires the same raw release, filtering, split, candidate set, and metric implementation.

Family	Sources summarized	Main protocol caveat	Exact match
MovieLens-1M	AMIM, DiffuRecSys, BSARec, HoloMambaRec	Rating filters, split details, and sampled versus full-catalog ranking vary	No
Yelp19/23	PAAC and LightGCN provide Yelp2018 references	Yelp2018 collaborative filtering is not the project’s local chronological Yelp 2019/2023 sequential snapshot	No
Amazon	ReSID, IDIOMoE, BLAIR, MARS, SIGMA, HM2Rec, MQSA-TED, Mamba4Rec, DuoRec, FMLP-Rec	Release year, category mapping, k -core filter, and candidate protocol vary	No

No verified paper in the selected primary literature reports the exact project-specific Yelp19 and Yelp23 snapshots under this repository’s rating- ≥ 4 , leave-two-out, full-catalog protocol. The Yelp rows below are therefore explicitly contextual rather than silently relabeled as Yelp19/23.

The notation is $R@K$ for Recall@ K , $H@K$ for Hit Ratio@ K , $N@K$ for NDCG@ K , and $M@K$ for MRR@ K . With one held-out positive item per user, $R@K$ and $H@K$ are numerically equivalent,

but each paper’s notation is retained. *Main result* means every Amazon score reported for the paper’s proposed method in its principal overall-result table; baseline rows copied into that paper are not duplicated unless the paper is itself a benchmark.

3.1 MovieLens-1M Sequential Recommendation

AMIM is a recent multi-interest sequential model evaluated on MovieLens-1M with Recall, NDCG, and MRR at 10 [?]. Its table is particularly relevant because it contains long-/short-interest and Transformer/RNN baselines. The scores below reproduce every MovieLens-1M row in the paper’s main comparison; they must not be treated as directly comparable to this project’s rating- ≥ 4 , chronological leave-two-out, full-catalog experiment without also reproducing AMIM’s preprocessing and candidate protocol.

Table 2: Published MovieLens-1M results reported by AMIM (Expert Systems with Applications, 2024).

Method	$R@10$	$N@10$	$M@10$
BERT4Rec	.2043	.1053	.0735
GRU4Rec	.2065	.1013	.0712
STAMP	.2506	.1376	.1035
SASRec	.2632	.1411	.1088
SASRecD	.2705	.1443	.1106
MIND	.2401	.1355	.1021
SINE	.2365	.1275	.0965
TAMIC	.2684	.1425	.1112
AMIM	.2787	.1612	.1278

DiffuRecSys uses chronological leave-two-out splitting and reports full-catalog next-item results at three cutoffs [?]. This protocol is close in form to the present project, although its rating filtering, optimization, and item universe must still be matched before making a direct claim. The table below reproduces every MovieLens-1M row from its main comparison.

Table 3: MovieLens-1M results from DiffuRecSys (2024).

Method	$H@5$	$H@10$	$H@20$	$N@5$	$N@10$	$N@20$
GRU4Rec	.0806	.1344	.2081	.0475	.0649	.0834
SASRec	.1078	.1810	.2745	.0681	.0918	.1156
BERT4Rec	.1308	.2219	.3354	.0804	.1097	.1384
STOSA	.1230	.1889	.2724	.0810	.1040	.1231
ACVAE	.1356	.2033	.3085	.0837	.1145	.1392
VSAN	.1220	.2016	.3015	.0751	.1007	.1257
MFGAN	.1275	.2086	.3166	.0778	.1040	.1309
CL4SRec	.1142	.1815	.2818	.0705	.0920	.1170
DuoRec	.1679	.2540	.3478	.1091	.1370	.1607
DiffuRecSys	.1957	.2843	.3958	.1319	.1605	.1886

BSARec, accepted at AAAI 2024, combines self-attention with a Fourier-domain inductive bias and evaluates all items with full-catalog cross-entropy [?]. Its MovieLens-1M result is $H@5 = .1944$, $H@10 = .2757$, $H@20 = .3884$, $N@5 = .1306$, $N@10 = .1568$, and $N@20 = .1851$. A later reproducibility study warns that padding and implementation details can move these scores, reinforcing the need to compare against a locally rerun baseline rather than only the published number [?].

HoloMambaRec is a 2026 selective-state recommender evaluated under a fixed 10-epoch training budget [?]. Table ?? reports its entire MovieLens-1M comparison. These values are useful efficiency-oriented references, not evidence that ten epochs is sufficient for every architecture.

Table 4: MovieLens-1M 10-epoch results from HoloMambaRec (2026).

Method	$H@10$	$N@10$
GRU4Rec	.1262	.0636
SASRec	.1361	.0730
HoloMambaRec	.1697	.0933

3.2 Yelp Results Under Distinct Protocols

Yelp dataset labels are especially overloaded. PAAC uses the standard implicit-feedback *Yelp2018* collaborative-filtering benchmark and reports complete-catalog top-20 metrics [?]. This is not the same corpus or task as this project’s local Yelp 2019 rating-filtered sequential snapshot, but it provides a recent graph-recommendation reference that is relevant to the added LightGCN branch. Table ?? reproduces the conventional-test rows in PAAC’s main comparison.

Table 5: Yelp2018 conventional-test results from PAAC (KDD 2024).

Method	$R@20$	$H@20$	$N@20$
LightGCN	.0591	.0614	.0483
Adapt- τ	.0724	.0753	.0603
SimGCL	.0720	.0748	.0594
PAAC	.0722	.0755	.0602

For an older but widely reused Yelp2018 reference point, the original LightGCN paper reports $R@20 = .0649$ and $N@20 = .0530$ under its own setup [?]. Differences between that row and PAAC’s rerun illustrate why the implementation and split must accompany every score. Neither Yelp2018 table is used to claim improvement by the present Yelp19 experiment.

3.3 Amazon Reviews 2023: Full-Catalog and Generative Results

ReSID evaluates ten 5-core Amazon Reviews 2023 subsets with chronological leave-two-out splitting [?]. These are the closest broad, recent category-level references found, although ReSID is a Semantic-ID generative recommender rather than an RL policy.

Table 6: ReSID main results on Amazon Reviews 2023 5-core subsets.

Subset	$R@5$	$R@10$	$N@5$	$N@10$
Musical_Instruments	.0417	.0645	.0273	.0346
Video_Games	.0597	.0927	.0396	.0501
Industrial_and_Scientific	.0340	.0512	.0218	.0273
Baby_Products	.0285	.0441	.0186	.0236
Arts_Crafts_and_Sewing	.0313	.0485	.0205	.0261
Sports_and_Outdoors	.0210	.0323	.0137	.0174

Subset	$R@5$	$R@10$	$N@5$	$N@10$
Toys_and_Games	.0261	.0392	.0174	.0216
Health_and_Household	.0208	.0317	.0140	.0175
Beauty_and_Personal_Care	.0230	.0357	.0150	.0191
Books	.0530	.0737	.0369	.0436

IDIOMoE, accepted at ICLR 2026, reports full-catalog results for three large Amazon Reviews 2023 collections [?]. The source calls the first collection *Beauty(23)*; it is left under that exact name because the paper does not disambiguate it as *All_Beauty* or *Beauty_and_Personal_Care*. Table ?? includes all methods in the paper’s large-catalog table.

Table 7: Full-catalog Amazon Reviews 2023 results from IDIOMoE.

Method	Beauty(23)		Books(23)		Toys(23)	
	$N@10$	$H@10$	$N@10$	$H@10$	$N@10$	$H@10$
SASRec	.0051	.0101	.0064	.0128	.0122	.0245
HSTU	.0130	.0247	.0211	.0410	.0149	.0332
ID Transformer	.0068	.0095	.0224	.0295	.0048	.0079
Text-Attr LLM	.0105	.0163	.0195	.0290	.0164	.0300
Item-LLM	.0082	.0119	.0174	.0261	.0079	.0148
IDIOMoE	.0119	.0228	.0224	.0419	.0186	.0361

BLaIR uses timestamp-based chronological splits on unfiltered Amazon Reviews 2023 and evaluates UniSRec with different semantic encoders [?]. It therefore has much larger catalogs than this project’s current All Beauty 5-core preprocessing. Table ?? reproduces all sequential-recommendation $N@10$ scores in its main encoder comparison.

Table 8: BLaIR/UniSRec $N@10$ on unfiltered Amazon Reviews 2023.

Semantic encoder	All_Beauty	Video_Games	Baby_Products
RoBERTa-large	.0177	.0113	.0070
Qwen3-Embedding-0.6B	.0224	.0126	.0072
Sup-SimCSE-RoBERTa-large	.0232	.0114	.0069
Sentence-T5-large	.0212	.0125	.0075
Qwen3-Embedding-4B	.0212	.0127	.0075
Qwen3-Embedding-8B	.0216	.0138	.0072
SFR-Embedding-Mistral	.0231	.0131	.0076
E5-Mistral-7B-Instruct	.0238	.0136	.0072
GritLM-7B	.0216	.0133	.0074
Gemini-Embedding-001	.0241	.0136	.0073
Text-Embedding-3-Large	.0237	.0135	.0078

3.4 Amazon Reviews 2018 and Legacy Sequential Benchmarks

MARS uses Amazon Reviews 2018 5-core sequences and chronological leave-one-out evaluation [?]. *Prime Pantry* is retained as reported and is not silently relabeled as *Grocery_and_Gourmet_Food*.

Table 9: MARS main results on Amazon Reviews 2018 5-core subsets.

Subset	$R@10$	$N@10$
Industrial_and_Scientific	.1533	.1086
Prime Pantry	.0928	.0591
Musical_Instruments	.1082	.0846
Arts_Crafts_and_Sewing	.1684	.1306
Office_Products	.1571	.1270

Table ?? adds recent Mamba recommenders and established sequential references. SIGMA is an AAAI 2025 Mamba model [?]; HM2Rec is an AAAI 2026 hyperbolic graph-Mamba model with a mixture-of-experts path [?]; MQSA-TED is a WSDM 2024 transition-distillation model [?]; and Mamba4Rec, DuoRec, and FMLP-Rec provide established references [?, ?, ?]. Dashes denote unreported metrics. HM2Rec’s high values come from sampled-candidate evaluation and must not be compared numerically with full-catalog rows.

Table 10: Legacy Amazon sequential results under each paper’s own protocol.

Model	Subset	$H@5$	$N@5$	$H@10$	$N@10$	$M@10$	$H@20/N@20$
SIGMA	Beauty	–	–	.0986	.0604	.0488	–
SIGMA	Sports	–	–	.0735	.0590	.0556	–
SIGMA	Games	–	–	.1627	.1088	.0924	–
HM2Rec	Books	–	–	.7874	.6053	–	.8487/.6209
HM2Rec	Toys	–	–	.6295	.4383	–	.7262/.4629
MQSA-TED	Beauty	.0752	.0534	.1039	.0627	–	.1435/.0726
MQSA-TED	Sports	.0455	.0320	.0643	.0380	–	.0906/.0446
MQSA-TED	Toys	.0834	.0600	.1130	.0696	–	.1503/.0789
Mamba4Rec	Beauty	–	–	.0812	.0451	.0362	–
Mamba4Rec	Video Games	–	–	.1152	.0603	.0438	–
DuoRec	Beauty	–	–	.0845	.0443	–	–
DuoRec	Clothing	–	–	.0302	.0148	–	–
FMLP-Rec	Beauty	.0398	.0258	.0632	.0333	–	.0958/.0415
FMLP-Rec	Sports	.0218	.0144	.0344	.0185	–	.0537/.0233
FMLP-Rec	Toys	.0456	.0317	.0683	.0391	–	.0991/.0468

3.5 Different Amazon Recommendation Tasks

The following results are useful references but are not sequential next-item results. JGCF performs user-item collaborative filtering on Amazon Books [?]. The Amazon Fashion study formulates recommendation as dense retrieval from a review query to a purchased product in an 826,402-item catalog [?].

Table 11: Published Amazon scores for non-sequential tasks.

Task/model	Subset	$R@10$	$N@10$	$R@20$	$N@20$	$R@50$	$N@50$
CF/JGCF	Books	.0796	.0671	.1191	.0798	.1949	.1019

Dense-retrieval model	Subset	$R@10$	$M@10$
BM25	Amazon_Fashion	.2600	.2500
Zero-shot MPNet	Amazon_Fashion	.4200	.3600
Fine-tuned MPNet	Amazon_Fashion	.6600	.5900

3.6 Coverage of the Requested Amazon Subsets

Table ?? audits every requested subset. *Not found* means that no verified category-specific score was found in the selected primary papers above; it does not prove that no result exists anywhere. Legacy labels such as *Beauty*, *Games*, or *Toys* are not automatically mapped to Amazon Reviews 2023 categories.

Table 12: Coverage audit for all requested Amazon subsets.

Requested subset	Verified source and comparability
All_Beauty	BLaIR, Amazon Reviews 2023, unfiltered full-catalog $N@10$
Amazon_Fashion	Dense retrieval only; not sequential recommendation
Appliances	Not found in the selected primary papers
Arts_Crafts_and_Sewing	ReSID (2023 5-core); MARS (2018 5-core)
Automotive	Not found in the selected primary papers
Baby_Products	ReSID (2023 5-core); BLaIR (2023 unfiltered)
Beauty_and_Personal_Care	ReSID (2023 5-core); IDIOMoE’s Beauty(23) kept separate
Books	ReSID and IDIOMoE (2023); JGCF/HM2Rec legacy protocols
Cell_Phones_and_Accessories	Not found in the selected primary papers
Clothing_Shoes_and_Jewelry	DuoRec reports legacy Clothing only
Digital_Music	Not found in the selected primary papers
Electronics	Not found in the selected primary papers
Gift_Cards	Not found in the selected primary papers
Grocery_and_Gourmet_Food	Not found; MARS Prime Pantry is not equivalent
Handmade_Products	Not found in the selected primary papers
Health_and_Household	ReSID, Amazon Reviews 2023 5-core
Health_and_Personal_Care	Not found in the selected primary papers
Home_and_Kitchen	Not found in the selected primary papers
Industrial_and_Scientific	ReSID (2023 5-core); MARS (2018 5-core)
Kindle_Store	Not found in the selected primary papers
Magazine_Subscriptions	Not found in the selected primary papers
Movies_and_TV	Not found in the selected primary papers
Musical_Instruments	ReSID (2023); MARS legacy protocol
Office_Products	MARS, Amazon Reviews 2018 5-core

Requested subset	Verified source and comparability
Patio_Lawn_and_Garden	Not found in the selected primary papers
Pet_Supplies	Not found in the selected primary papers
Software	Not found in the selected primary papers
Sports_and_Outdoors	ReSID (2023); SIGMA/MQSA-TED/FMLP-Rec legacy Sports
Subscription_Boxes	Not found in the selected primary papers
Tools_and_Home_Improvement	Not found in the selected primary papers
Toys_and_Games	ReSID and IDIOMoE (2023); HM2Rec/MQSA-TED/FMLP-Rec legacy Toys
Video_Games	ReSID and BLaIR (2023); Mamba4Rec legacy Video Games
Unknown	Not a standard benchmark subset; no verified score

3.7 Comparison Rule for This Project

The present All Beauty experiment uses a project-specific 5-core catalog with 1,620 users and 6,982 items and performs full-catalog ranking. Its $R@10 = .15432$ therefore must not be described as outperforming BLaIR, BERT4Rec, HM2Rec, or another paper solely because the number is larger. A defensible comparison requires the same raw Amazon release, category mapping, filtering, split, seen-item masking, candidate catalog, and metric code. The recommended reporting order is: (1) reproduce SASRec and Mamba4Rec in this project’s data loader; (2) report the proposed model and those baselines under the identical full-catalog evaluator; and (3) retain the literature tables in this section only as external reference points.

4 Method

4.1 Problem Formulation

Let $\mathcal{U}$ be users and $\mathcal{I}$ the catalog, with $N = |\mathcal{I}|$. User interactions are ordered:

$$\mathcal{H}_u = (i_1, i_2, \dots, i_{T_u}). \quad (1)$$

At transition t , the state is prefix $s_t = (i_1, \dots, i_t)$ and logged next item i_{t+1} is positive. The agent scoring distributions are

$$\pi_L(a \mid s_L), \quad \pi_S(a \mid s_S), \quad \pi_P(a \mid s_L), \quad \pi_C(a \mid s_L, s_S, \pi_P). \quad (2)$$

4.2 Pipeline Overview

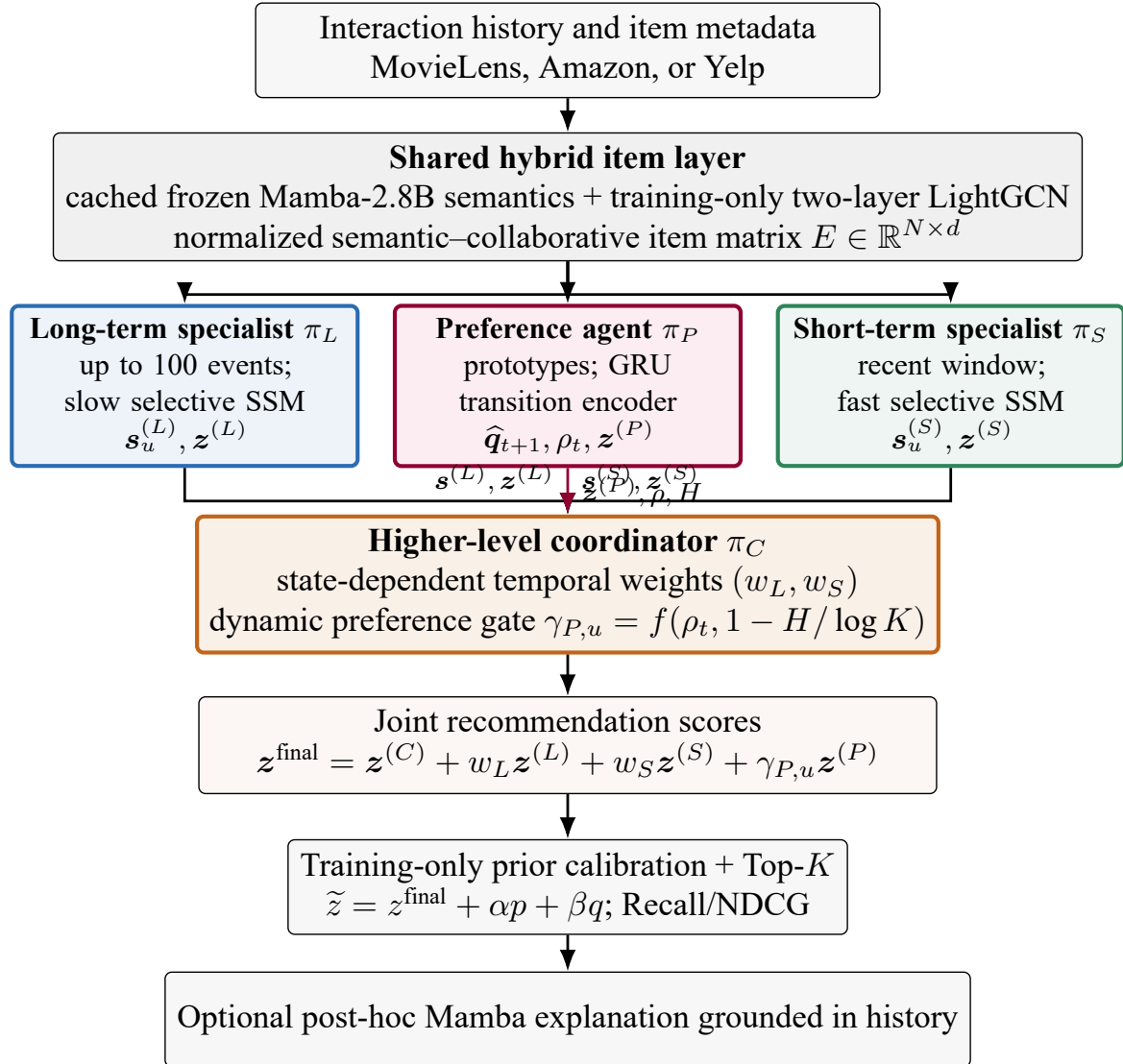


Figure 1: Hierarchical cooperative multi-agent Mamba ranking architecture. Solid arrows show the inference path.

The hierarchy assigns complementary responsibilities rather than making the rankers vote independently. The two temporal specialists extract preferences at different time scales, and the higher-level coordinator decides their state-dependent contribution to one deployed ranking. During joint training, the coordinator and all specialists receive listwise ranking gradients; during inference, only the solid forward path is executed. Each colored agent box owns its own LoRA route; no adapter parameters are shared across agent routes.

4.3 Data and Chronological Splitting

Every record becomes (u, i, τ, t_i) : user, item, timestamp, and item text. IDs become contiguous integers. Users need at least five positive events by default. MovieLens and Yelp use rating at least four.

Table 13: Dataset interfaces and item text.

Argument	Interactions	Item text
movielens	ratings	title and genres
amazon-<category>	Amazon Reviews 2023	title, subtitle, main category, features, description
amazon:<exact-config>	exact HF config	review/meta join by <code>parent_asin</code>
yelp19, yelp23	local Yelp JSON/table	name, categories, location, attributes
synthetic	deterministic events	short names for tests

Yelp is license-gated and requires a local path. Leave-two-out splitting is

$$\mathcal{H}_u^{\text{train}} = (i_1, \dots, i_{T_u-2}), \quad (3)$$

$$i_u^{\text{valid}} = i_{T_u-1}, \quad (4)$$

$$i_u^{\text{test}} = i_{T_u}. \quad (5)$$

Training prefixes produce $((i_1, \dots, i_k), i_{k+1})$. Reservoir sampling caps transitions at `MAX_TRANSITIONS`.

4.4 Cached Mamba Semantics

The frozen `state-spaces/mamba-2.8b-hf` backbone encodes metadata after the prefix `Preference-aware product representation:`. The Amazon suite caps the prompted input at 32 tokens. Prompt tokens condition the causal hidden states but are excluded from content mean pooling:

$$e_i^{\text{text}} = \text{MeanPool}(\text{Mamba}_{\text{frozen}}(t_i)) \in \mathbb{R}^{d_m}. \quad (6)$$

The prompt, item texts, dataset name, catalog size, and token cap determine the cache identity, preventing reuse of incompatible vectors. A shared bridge gives

$$e_i = \text{norm}(W_p e_i^{\text{text}}), \quad W_p \in \mathbb{R}^{d \times d_m}, \quad d = 128. \quad (7)$$

4.5 Training-Only LightGCN Fusion

The current implementation enables a two-layer LightGCN branch by default [?]. It constructs an undirected bipartite graph only from unique user-item pairs in `train_by_user`; validation and test edges are excluded. Let A be its adjacency matrix, D its degree matrix, and $X^{(0)} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|) \times d}$ trainable node embeddings. The code performs symmetric normalized propagation without feature transforms or nonlinearities:

$$X^{(\ell+1)} = D^{-1/2} A D^{-1/2} X^{(\ell)}, \quad \ell \in \{0, 1\}, \quad (8)$$

$$\bar{X} = \frac{1}{3} \sum_{\ell=0}^2 X^{(\ell)}. \quad (9)$$

Writing e_i^{gcn} for item i 's row of $\bar{X}$, the actual vector consumed by both specialists and by catalog scoring is

$$e_i = \text{norm}(\text{norm}(W_p e_i^{\text{text}}) + \text{norm}(e_i^{\text{gcn}})). \quad (10)$$

Thus the graph is a shared collaborative item encoder, not a fifth agent. It is optimized with the item projection during specialist warm-up and joint training, frozen during coordinator warm-up, and recomputed once per batch for reuse by histories and candidates. The optional `--graph-device` places graph parameters and edges on a second GPU; `--no-use-graph-embeddings` provides the semantic-only ablation.

4.6 Agent-Specific LoRA

For input $\mathbf{x}$, agent j owns

$$\text{LoRA}_j(\mathbf{x}) = \frac{\alpha}{r} B_j A_j \text{Dropout}(\mathbf{x}). \quad (11)$$

The generic single-dataset launcher defaults to $r = 8$ and $\alpha = 16$, while the ver4 Amazon multi-LoRA suite used in this report sets $r = 16$ and $\alpha = 32$; both use dropout 0.05. A_j uses Kaiming initialization and B_j starts at zero. The complete routing table is shown in Table ??.

Table 14: Disjoint LoRA routes in the ver4 multi-LoRA system.

Agent	LoRA residuals	Count
Long-term	candidate, delta, gate, output	4
Short-term	candidate, delta, gate, output	4
Preference-transition	GRU-state transition, prototype-mixture state	2
Coordinator	temporal mixture, fused state, score state	3
Total	disjoint automatically routed adapters	13

The adapter bank operates only in the compact recommender. Cached Mamba-2.8B features are immutable inputs, while the shared projection, LightGCN embeddings, preference prototypes, GRU and task heads are ordinary trainable recommender parameters. Thus “multi-LoRA” denotes agent-specific low-rank residual routes; it does not imply adaptation of the Mamba-2.8B backbone.

4.7 Selective-State Specialists

For $j \in \{L, S\}$:

$$\delta_t^{(j)} = \text{softplus} \left(\mathbf{b}_\delta^{(j)} + \text{LoRA}_\delta^{(j)}(\mathbf{x}_t) \right), \quad (12)$$

$$\mathbf{a}_t^{(j)} = \exp(-\delta_t^{(j)}), \quad (13)$$

$$\tilde{\mathbf{h}}_t^{(j)} = \tanh \left(\mathbf{x}_t + \text{LoRA}_{\text{candidate}}^{(j)}(\mathbf{x}_t) \right), \quad (14)$$

$$\mathbf{h}_t^{(j)} = \mathbf{a}_t^{(j)} \odot \mathbf{h}_{t-1}^{(j)} + (1 - \mathbf{a}_t^{(j)}) \odot \tilde{\mathbf{h}}_t^{(j)}, \quad (15)$$

$$\mathbf{g}_t^{(j)} = \text{sigmoid} \left(\text{LoRA}_{\text{gate}}^{(j)}(\mathbf{x}_t) \right), \quad (16)$$

$$\mathbf{o}_t^{(j)} = \mathbf{g}_t^{(j)} \odot \mathbf{h}_t^{(j)} + (1 - \mathbf{g}_t^{(j)}) \odot \mathbf{x}_t. \quad (17)$$

Small δ_t preserves history; large values adopt the current item. Long sees up to 100 events at initial time scale 0.08; short sees 10 at 0.5. Logits are

$$z_i^{(j)} = \exp(\tau_j) \langle \mathbf{s}_u^{(j)}, \mathbf{e}_i \rangle. \quad (18)$$

4.8 Preference Prototype Discovery and Transition Prediction

The preference agent does not require manually labeled preference categories. It learns K prototype vectors $P = [\mathbf{p}_1; \dots; \mathbf{p}_K] \in \mathbb{R}^{K \times d}$ shared by every user in one dataset. For normalized item vector $\mathbf{e}_t$, its soft pseudo-preference is

$$\mathbf{q}_t = \text{softmax} \left(\frac{\text{norm}(\mathbf{e}_t) \text{norm}(P)^\top}{T_p} \right), \quad \mathbf{q}_t \in \Delta^{K-1}, \quad (19)$$

where $T_p = 0.2$ by default. These assignments are latent pseudo-preferences, not human-authored genres or categories.

A GRU reads $(q_1, \dots, q_t)$. If h_t^G is the last valid GRU state, an agent-specific LoRA residual produces

$$h_t^{(P)} = \text{LayerNorm}(h_t^G + \text{LoRA}_{\text{transition}}(h_t^G)). \quad (20)$$

The LoRA residual begins as an exact no-op because its second matrix is zero-initialized. The current and predicted-next distributions and the transition probability are

$$q_t^{\text{current}} = q_t, \quad (21)$$

$$\hat{q}_{t+1} = \text{softmax}(W_n h_t^{(P)} + b_n), \quad (22)$$

$$\ell_t^{\text{change}} = W_c[h_t^{(P)}; q_t] + b_c, \quad (23)$$

$$\rho_t = \text{sigmoid}(\ell_t^{\text{change}}). \quad (24)$$

The next-preference state and candidate logits are

$$v_t = \hat{q}_{t+1} \text{norm}(P), \quad (25)$$

$$s_u^{(P)} = \text{norm}(v_t + \text{LoRA}_{\text{state}}(v_t)), \quad (26)$$

$$z_i^{(P)} = \exp(\tau_P) \langle s_u^{(P)}, e_i \rangle. \quad (27)$$

The state adapter also begins as a no-op. The raw change logit is used for AMP-safe binary cross entropy with logits; ρ_t is used for analysis and for the coordinator’s context-dependent preference gate.

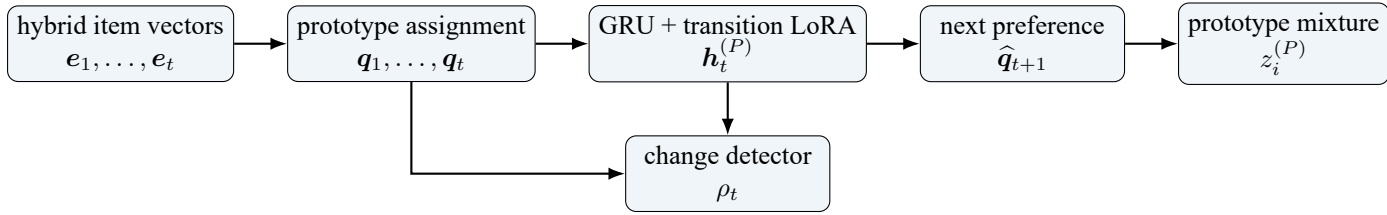


Figure 2: Preference-transition branch in ver4. Soft assignments expose a latent preference trajectory; the branch predicts both the next distribution and the probability of a transition before contributing candidate scores.

4.9 Preference Training Objectives

The target item’s detached assignment q^+ supervises the next distribution:

$$\mathcal{L}_{\text{next}} = D_{\text{KL}}(q^+ \| \hat{q}_{t+1}). \quad (28)$$

Transition strength is a soft target derived from Jensen–Shannon divergence:

$$m = \frac{1}{2}(q_t^{\text{current}} + q^+), \quad (29)$$

$$y_t^{\text{change}} = \frac{\frac{1}{2}D_{\text{KL}}(q_t^{\text{current}} \| m) + \frac{1}{2}D_{\text{KL}}(q^+ \| m)}{\log 2}, \quad (30)$$

$$\mathcal{L}_{\text{change}} = \text{BCEWithLogits}(\ell_t^{\text{change}}, y_t^{\text{change}}). \quad (31)$$

If $\bar{q} = B^{-1} \sum_b q_b^+$, two anti-collapse terms are

$$\mathcal{L}_{\text{balance}} = D_{\text{KL}}(\bar{q} \| \text{Uniform}(K)), \quad (32)$$

$$\mathcal{L}_{\text{separate}} = \frac{1}{K^2} \| \text{norm}(P) \text{norm}(P)^\top - I \|_F^2. \quad (33)$$

The default auxiliary objective is

$$\mathcal{L}_P = 0.2\mathcal{L}_{\text{next}} + 0.1\mathcal{L}_{\text{change}} + 0.01\mathcal{L}_{\text{balance}} + 0.01\mathcal{L}_{\text{separate}}. \quad (34)$$

4.10 Coordinator and Final Ranking

$$\mathbf{q}_u = [\mathbf{s}_u^{(L)}; \mathbf{s}_u^{(S)}], \quad (35)$$

$$[w_L, w_S] = \text{softmax}(\text{LoRA}_{\text{mix}}(\mathbf{q}_u)), \quad (36)$$

$$\mathbf{s}_u^{(C)} = \text{norm} \left(w_L \mathbf{s}_u^{(L)} + w_S \mathbf{s}_u^{(S)} + \text{LoRA}_{\text{state}}(\mathbf{q}_u) \right), \quad (37)$$

$$c_t^P = 1 - H(\hat{\mathbf{q}}_{t+1}) / \log K, \quad (38)$$

$$\gamma_{P,u} = \text{sigmoid}(g_P + f_g[\rho_t; c_t^P]), \quad (39)$$

$$z_i^{\text{final}} = z_i^{(C)} + w_L z_i^{(L)} + w_S z_i^{(S)} + \gamma_{P,u} z_i^{(P)}. \quad (40)$$

The base scalar g_P is initialized so that $\gamma_{P,u} = 0.2$, and f_g starts at zero. Thus the dynamic gate initially reproduces the previous global weight, then learns user-dependent calibration from transition probability and next-preference confidence while coordinator warm-up freezes the encoder.

4.11 Training-Only Catalog Calibration

The neural coordinator can be augmented by two deterministic catalog priors. They are estimated strictly from $\mathcal{H}_u^{\text{train}}$ and therefore introduce no validation or test interactions into the feature construction. Let c_i be the number of occurrences of item i in all training histories. The standardized log-popularity feature is

$$p_i = \frac{\log(1 + c_i) - \mu_p}{\max(\sigma_p, 10^{-6})}. \quad (41)$$

For adjacent training events, let $n_{a,b}$ count transitions from item a to item b and define

$$q_{a,b} = \log(1 + n_{a,b}). \quad (42)$$

If ℓ_u is the last observed item in the inference history, the deployed full-catalog score is

$$\tilde{z}_{u,i} = z_{u,i}^{\text{final}} + \alpha p_i + \beta q_{\ell_u,i}, \quad q_{\ell_u,i} = 0 \text{ for unseen transitions}. \quad (43)$$

Seen non-target items are masked after calibration. The coefficients α and β are selected using validation ranking only; the held-out test set is evaluated once after the choice is fixed. A negative α suppresses over-popular items, while a positive β reinforces locally plausible next-item transitions. This calibration is not an agent and does not update model weights. Both coefficients default to zero, so existing runs retain the original neural ranking exactly.

5 Training Strategy

5.1 Training Targets and Hard Negatives

For every training prefix ending at t , up to $H = 3$ future items from the training sequence receive exponentially decayed weights:

$$\omega_h = \frac{\eta^{h-1}}{\sum_{r=1}^{H_u} \eta^{r-1}}, \quad \eta = 0.5, \quad h \leq H_u \leq H. \quad (44)$$

Validation and test targets are never used to construct these labels. A random pool four times larger than the required negative set is scored by the current coordinator. Items occurring in that user's training history are masked, and the highest-scoring remaining items become model-aware hard negatives. For agent j , the soft listwise loss is

$$\mathcal{L}_{\text{rank}}^{(j)} = - \sum_{h=1}^{H_u} \omega_h \log \text{softmax}(\mathbf{z}^{(j)})_{i_{t+h}}. \quad (45)$$

The bounded transition set is regenerated with a new deterministic seed every epoch, so large datasets expose different prefixes without increasing one epoch's memory budget.

5.2 Stage 1: Specialist Warm-Up

The coordinator is frozen:

$$\mathcal{L}_{\text{specialist}} = \mathcal{L}_{\text{rank}}^{(L)} + \mathcal{L}_{\text{rank}}^{(S)} + \mathcal{L}_{\text{rank}}^{(P)} + \mathcal{L}_P. \quad (46)$$

5.3 Stage 2: Coordinator Warm-Up

Specialists and projection are frozen:

$$\mathcal{L}_{\text{coord}} = \mathcal{L}_{\text{rank}}^{(C)}. \quad (47)$$

5.4 Stage 3: Joint Ranking and Preference Alignment

The preference state is aligned with immediate target item vectors using an in-batch multi-positive contrastive objective. Samples sharing the same target are all treated as positives rather than false negatives:

$$\mathcal{P}_b = \{r : i_r^+ = i_b^+\}, \quad (48)$$

$$\mathcal{L}_{\text{contrast}} = -\frac{1}{B} \sum_b \log \frac{\sum_{r \in \mathcal{P}_b} \exp(\text{sim}(\mathbf{s}_b^{(P)}, \mathbf{e}_r^+)/\tau)}{\sum_r \exp(\text{sim}(\mathbf{s}_b^{(P)}, \mathbf{e}_r^+)/\tau)}, \quad (49)$$

$$\mathcal{L}_{\text{joint}} = \frac{1}{4} \sum_{j \in \{L, S, P, C\}} \mathcal{L}_{\text{rank}}^{(j)} + \mathcal{L}_P + 0.05 \mathcal{L}_{\text{contrast}}. \quad (50)$$

The previous cosine-squared long/short specialization penalty is removed: different history windows and decay time scales already provide structural specialization, while forced orthogonality can discard preference information shared across temporal scales.

5.5 Gradients and Optimization

Coordinator ranking gradients flow through fusion into both specialist paths, and every specialist receives its own listwise ranking loss.

Table 15: Optimization used by the ver4 Amazon multi-LoRA suite.

Stage	Epochs	Learning rate	Trainable components
Specialists	4	1×10^{-4}	long, short, preference, projection, LightGCN
Coordinator	4	1×10^{-4}	coordinator and preference score gate
Joint ranking	10	5×10^{-5}	all recommender components

AdamW uses weight decay 10^{-5} . CUDA uses FP16 autocast, GradScaler, and gradient clipping at 1.0. The Amazon suite uses batch size 128, evaluation batch size 64, dataset-specific candidate counts and validation intervals, and caps the effective interval at one epoch so every epoch is validated. A ReduceLROnPlateau scheduler halves the coordinator or joint learning rate after two non-improving checks, and joint training stops after six non-improving checks. Because the Amazon launcher sets SAVE_MODEL_WEIGHTS=0, the best validation state is retained in memory rather than written as a checkpoint; final metrics are computed only after restoring that state.

6 Full-Catalog Evaluation and Throughput

Evaluation projects the complete catalog once even though the reported Amazon runs train with bounded hard-negative slates:

$$E = \text{HybridProject}(E_{\text{text}}, E_{\text{gcn}}) \in \mathbb{R}^{N \times d}, \quad Z^{(j)} = S^{(j)} E^\top \in \mathbb{R}^{B \times N}. \quad (51)$$

Each agent applies its learned temperature, and the coordinator combines $Z^{(L)}$, $Z^{(S)}$, $Z^{(P)}$, and its own adapted-state score to obtain Z^{final} . Matrix multiplication avoids a $B \times N \times d$ expansion. Popularity and transition vectors are then added by broadcasting and sparse row updates to obtain $\tilde{Z}$; they do not require another model forward pass. Seen non-target items receive $-\infty$.

$$\text{rank}_u = \sum_{i=1}^N \mathbb{I}[\tilde{z}_{u,i} \geq \tilde{z}_{u,i+}], \quad (52)$$

$$\text{Recall@K} = \frac{1}{|\mathcal{U}|} \sum_u \mathbb{I}[\text{rank}_u \leq K], \quad (53)$$

$$\text{NDCG@K} = \frac{1}{|\mathcal{U}|} \sum_u \frac{\mathbb{I}[\text{rank}_u \leq K]}{\log_2(\text{rank}_u + 1)}. \quad (54)$$

The code reports $K = 5, 10$, users/s, and scores/s.

Throughput comes from cached Mamba vectors, shared semantics, linear selective recurrences, a GRU over 64-dimensional preference assignments, capped history, bounded listwise candidate sets with model-aware hard negatives, FP16, GEMM ranking, sparse prior updates, and delayed generator loading.

7 Recommendation Explanation

For sampled test users, Top-1, ten recent item texts, and coordinator weights form:

Explain this recommendation in one concise sentence using only the supplied history.
 Long-term agent weight=<wL>; short-term agent weight=<wS>.
 History: <recent item texts>.
 Recommended item: <recommended item text>. Reason:

When enabled, Mamba greedily generates at most 40 tokens after ranking. The explanation is grounded but post-hoc; it is not a causal proof and does not affect training or ranking. The Amazon suite used in this report disables reason generation.

8 Experimental Design

Required ablations include removal of each of the four agents, identical versus 100/10 windows, fixed versus learned fusion, random versus model-hard negatives, one-step versus future soft labels, fixed versus per-epoch prefix sampling, disjoint versus shared LoRA routes, and alternative LoRA ranks. Catalog calibration must be ablated as neural-only, popularity-only, transition-only, and combined scoring, with α and β selected on validation for every dataset. Semantic features should compare Mamba vectors, random vectors, and ID embeddings. Sequence baselines should include GRU and Transformer under matched capacity. Random negatives should be compared with hard negatives.

Beyond Recall/NDCG, useful measures include coverage, novelty, diversity, results by history length, coordinator-weight distribution, specialist-state cosine similarity, transitions/s, users/s, scores/s, peak GPU memory, and explanation latency. Because the priors can change exposure concentration, coverage and popularity-stratified Recall should accompany aggregate Recall.

8.1 Completed ver4 Multi-LoRA Result

Table ?? reports the completed multi-LoRA run present in the version-local output directory at writing time. It uses Amazon Reviews 2023 All Beauty with 1,620 users, 6,982 items, and 11,744 training interactions, chronological leave-two-out splitting, one held-out item per user, and full-catalog evaluation. Consequently, Recall and Hit Ratio are identical. The run uses all 13 LoRA adapters with $r = 16$, $\alpha = 32$, and dropout 0.05; four specialist-stage, four coordinator-stage, and ten joint-stage epochs; 64 training candidates; and validation-selected catalog coefficients $\alpha_p = -0.25$ and $\beta = 4.0$.

Table 16: Completed ver4 Preference-Transition Multi-LoRA result.

Split	Run	$R@5$	$R@10$	$H@5$	$H@10$	$N@5$	$N@10$
Validation	20260830_011259	.09012	.10123	.09012	.10123	.08258	.08610
Test	20260830_011259	.11358	.11914	.11358	.11914	.10254	.10431

This is an engineering measurement, not a paper-to-paper victory claim. Published results may use different Amazon releases, category definitions, core filters, and candidate protocols. A controlled comparison requires rerunning baselines through the same loader and evaluator.

The active Beauty and Personal Care job is not included because its log does not yet contain final validation and test artifacts. Results from predecessor versions are deliberately excluded from the ver4 result table.

9 Implementation and Reproducibility

Table 17: Independent implementation files.

File	Responsibility
run_mamba_rl.sh	single-dataset multi-LoRA launcher
run_amazons_full_rl.sh	Amazon multi-LoRA suite and subset settings
src/data_mamba_rl.py	dataset adapters
src/model_mamba_rl.py	four agents and disjoint LoRA routing
src/train_mamba_rl.py	training, evaluation, explanations
tests/test_mamba_rl.py	tests
outputs_mamba_rl/	artifacts

```
bash run_mamba_rl.sh movielens-1m 0
bash run_mamba_rl.sh movielens-20m 0
bash run_mamba_rl.sh amazon-all-beauty 0
bash run_mamba_rl.sh amazon-movies-and-tv 0
bash run_mamba_rl.sh yelp19 0 /data/yelp-2019
bash run_mamba_rl.sh yelp23 0 /data/yelp-2023
```

Environment variables include `CACHE_DIR`, `BATCH_SIZE`, `EVAL_BATCH_SIZE`, `SPECIALIST_EPOCHS`, `COORDINATOR_EPOCHS`, `JOINT_EPOCHS`, `VALIDATE_EVERY_STEPS`, `MONITOR_METRIC`, `EARLY_STOPPING`, `LR_PATIENCE`, `MAX_TRANSITIONS`, `SEED`, `POPULARITY_ALPHA`, `TRANSITION_BETA`, `USE_GRAPH_EMBEDDING`, `GRAPH_DEVICE`, `TARGET_RECALL_AT_10`, `EXPERIMENT_NOTE`, `LORA_RANK`, `LORA_ALPHA`, `LORA_DROPOUT`, `PREFERENCE_COUNT`, `PREFERENCE_HIDDEN`, `PREFERENCE_TEMPERATURE`, `PREFERENCE_SCORE_WEIGHT`, `PREFERENCE_COEF`, `PREFERENCE_TRANSITION_COEF`, `PREFERENCE_BALANCE`, and `PREFERENCE_SEPARATION_COEF`. The version-local Amazon suite is

```
cd /workspace/P78123011/MediaTek_2026_project/ver4
bash run_amazons_full_rl.sh
```

A CPU smoke test is

```
python -m src.train_mamba_rl \
  --dataset synthetic --device cpu --skip-mamba \
  --no-generate-reasons --max-transitions 32 --batch-size 8
```

The Amazon suite deliberately disables weight persistence. Its outputs are `train_<run-id>.log`, `subset score JSON`, `loss.png`, `metrics.json`, `recommendations.json`, and `preference_analysis.js` under `outputs_mamba_rl/amazons_lora/<subset>/rl_<timestamp>/`. Every log begins with `EXPERIMENT_NOTE` and the complete serialized `EXPERIMENT_CONFIG`; calibrated runs additionally record `ADAPTATION_MODE mode=multi_lora`, `ADAPTER_ROUTES`, `CATALOG_PRIORS`, vector provenance, an optional resumed checkpoint, `VALID_BEST`, `TEST_FINAL`, and `TARGET_RESULT`. Legacy training paths do not import this pipeline.

10 Limitations

1. Static logs contain no action-dependent user response, so the current multi-LoRA system is an offline sequential ranker rather than a complete long-horizon MDP.
2. Random negatives may be unexposed; no propensity correction is used.
3. Model-mined negatives reduce but do not eliminate the mismatch between sampled training and full-catalog evaluation.
4. Popularity and first-order transition counts are simple, dataset-dependent priors and may not transfer across domains or temporal shifts.
5. Calibrated gains must be separated from gains in the learned neural ranker; the priors are deterministic post-training score corrections.
6. Explanations are post-hoc and not optimized for faithfulness.
7. Latent preference IDs have no guaranteed human semantic label and may permute across independent runs or datasets.
8. A high predicted transition probability identifies a learned embedding shift, not a causal change in a person's underlying taste.
9. Mamba-2.8B supplies frozen item semantics; the agent-specific LoRA bank adapts compact recommendation modules rather than the language-model backbone.

Recall/NDCG gains therefore mean improved offline ranking, not proven long-term satisfaction.

11 Future Research

Reward-adaptive memory can use

$$\delta_t = f_\theta(\mathbf{x}_t, r_{t-1}, \sigma_{t-1}). \quad (55)$$

Offline RL can add propensity weighting or

$$\mathcal{L}_{\text{CQL}} = \log \sum_a \exp Q(s, a) - Q(s, a_{\text{logged}}). \quad (56)$$

A simulator can support centralized critics and retention reward. Item selection and reasoning can be jointly optimized with

$$r = \lambda_1 r_{\text{rank}} + \lambda_2 r_{\text{grounding}} + \lambda_3 r_{\text{consistency}} + \lambda_4 r_{\text{diversity}}. \quad (57)$$

12 Conclusion

The pipeline combines Mamba item semantics, linear selective-state modeling, a GRU preference-transition model, and 13 agent-specific LoRA adapters without fine-tuning or duplicating the Mamba-2.8B backbone. The frozen backbone establishes a reusable semantic catalog; long and short agents model different time scales; the preference agent predicts latent preference transitions; and the coordinator learns context-dependent fusion. Staged multi-step listwise training establishes the specialists, calibrates coordination, and jointly adapts the multi-LoRA recommender. Training-only popularity and transition priors provide a leakage-controlled calibration layer whose effect remains explicitly separable from learned model quality. Periodic validation, best-state restoration, auditable experiment logs, and full-catalog evaluation make ranking quality and systems cost measurable.

References

- [1] E. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [2] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *COLM*, 2024.
- [3] M. M. Afsar, T. Crump, and B. Far. Reinforcement Learning Based Recommender Systems: A Survey. *ACM Computing Surveys*, 2022.
- [4] G. Zheng, F. Zhang, Z. Zheng, et al. DRN: A Deep Reinforcement Learning Framework for News Recommendation. *The Web Conference*, 2018.
- [5] M. Chen, A. Beutel, P. Covington, et al. Top-K Off-Policy Correction for a REINFORCE Recommender System. *WSDM*, 2019.
- [6] E. Ie, V. Jain, J. Wang, et al. Reinforcement Learning for Slate-Based Recommender Systems. *arXiv:1905.12767*, 2019.
- [7] A. L. Silva, D. Parra, and R. T. Icarte. On the Unexpected Effectiveness of Reinforcement Learning for Sequential Recommendation. *ICML*, 2024.
- [8] C. Liu, J. Lin, J. Wang, et al. Mamba4Rec: Towards Efficient Sequential Recommendation with Selective State Space Models. *arXiv:2403.03900*, 2024.
- [9] Z. Liu, Q. Liu, Y. Wang, et al. SIGMA: Selective Gated Mamba for Sequential Recommendation. *AAAI*, 2025.
- [10] Y. Liang, Z. Zhang, Y. Zhu, et al. Rethinking Generative Recommender Tokenizer: Recsys-Native Encoding and Semantic Quantization Beyond LLMs. *arXiv:2602.02338*, 2026.
- [11] R. Shirkavand, X. Wei, C. Wang, et al. Catalog-Native LLM: Speaking Item-ID Dialect with Less Entanglement for Recommendation. *ICLR*, 2026.
- [12] Y. Hou, J. Li, X. Fu, et al. Bridging Language and Items for Retrieval and Recommendation: Benchmarking LLMs as Semantic Encoders. *arXiv:2403.03952v2*, 2026.

- [13] H. Kim, J. Kim, M. Choi, S. Lee, and J. Lee. MARS: Matching Attribute-aware Representations for Text-based Sequential Recommendation. *CIKM*, 2024.
- [14] Y. Liu, L. Qi, X. Mao, et al. Hyperbolic-Enhanced Mixture-of-Experts Mamba for Sequential Recommendation. *AAAI*, 2026.
- [15] T. Zhu, Y. Shi, Y. Zhang, Y. Wu, F. Mo, and J.-Y. Nie. Collaboration and Transition: Distilling Item Transitions into Multi-Query Self-Attention for Sequential Recommendation. *WSDM*, 2024.
- [16] R. Qiu, Z. Huang, H. Yin, and Z. Wang. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. *WSDM*, 2022.
- [17] K. Zhou, H. Yu, W. X. Zhao, and J. R. Wen. Filter-Enhanced MLP Is All You Need for Sequential Recommendation. *The Web Conference*, 2022.
- [18] J. Guo, L. Du, X. Chen, et al. On Manipulating Signals of User–Item Graph: A Jacobi Polynomial-Based Graph Collaborative Filtering. *KDD*, 2023.
- [19] M. Pandey. Domain-Adaptive and Scalable Dense Retrieval for Content-Based Recommendation. *arXiv:2602.00899*, 2026.
- [20] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *SIGIR*, 2020.
- [21] Y. Cheng, Y. Fan, Y. Wang, and X. Li. Accurate Multi-Interest Modeling for Sequential Recommendation with Attention and Distillation Capsule Network. *Expert Systems with Applications*, vol. 243, article 122887, 2024.
- [22] M. Cai, L. Chen, Y. Wang, H. Bai, P. Sun, L. Wu, M. Zhang, and M. Wang. Popularity-Aware Alignment and Contrast for Mitigating Popularity Bias. *KDD*, 2024.
- [23] S. Zolghadr, O. Winther, and P. Jeha. Generative Diffusion Models for Sequential Recommendations. *RecSys 2024 Workshop / arXiv:2410.19429*, 2024.
- [24] Y. Shin, J. Choi, H. Wi, and N. Park. An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention. *AAAI*, 2024.
- [25] J. Hutter, H. C. Bakker, S. Fris, M. Bernardy, and Y. Liu. A Systematic Reproducibility Study of BSARec for Sequential Recommendation. *arXiv:2512.17442*, 2025.
- [26] A. Parthasarathy, A. M. Kirubakaran, V. Punniyamoorthy, et al. Scalable Sequential Recommendation under Latency and Memory Constraints. *arXiv:2601.08360*, 2026.